Mesh Optimisation via Scientific Machine Learning for Multiphysics Simulation

Numerical Analysis for Partial Differential Equations Type B Project Report

Pietro Caminiti, Dario Mazzotti, Leonardo Zuccher

Mathematical Engineering - Politecnico di Milano

Tutors: Prof. P.F. Antonietti, Dott. N. De March

July 2026

Abstract

The accuracy and cost of a finite element method are strongly affected by the computational mesh. Classical adaptive strategies improve the approximation by refining or moving the mesh, but relocation-based approaches usually require the solution of an additional nonlinear meshing problem and rely on monitor functions that approximate the true discretization error. This project studies and reproduces the G-Adaptivity method and accompanying `g-adaptivity` codebase introduced by Rowbottom et al. [1], in which a graph neural network is trained to relocate mesh nodes so as to reduce the finite element error of the underlying PDE solution. Starting from the Poisson benchmark on the unit square, the replication was extended to the full experimental suite that is reproducible from the released code: Poisson problems on convex and non-convex domains (square, convex polygon, L-shape, H-shape), the time-dependent Burgers equation in both the fixed-horizon and rollout evaluation protocols, and a Navier–Stokes flow past a cylinder. The final campaign consists of 73 runs executed on the DMAT Student PoliMi cluster with the paper-exact protocol described further on, with generally five independent training seeds per learned model. The qualitative claims of the reference paper are reproduced on every benchmark: the learned G-adaptive model matches or outperforms the Monge–Ampère (MA) baseline in L^2 error reduction while relocating meshes one to two orders of magnitude faster, and the failure regimes of MA on non-convex domains and multiply connected domains are confirmed. Quantitatively, most error-reduction values agree with the paper within the offset induced by regenerating the random datasets. The replication also produces findings that the paper does not report: the main is a seed-dependent training instability of the diffusion-based deformer on the Navier–Stokes and Burgers benchmarks. Particular attention is given to reproducibility and to the debugging steps required to expose the given code hidden failures, such as compatibility issues or code bugs related to PDE loss.

Contents

1 Introduction **5**

2 Mathematical and numerical background **5**
2.1 The Poisson model problem . . . 6
2.2 Weak formulation and finite element discretization . . . 6
2.3 Mesh relocation and r-adaptivity . . . 7
2.4 Classical Monge–Ampère mesh movement . . . 7
2.5 Other reference problems . . . 8
2.6 Error reduction and mesh quality metrics . . . 8

3 The G-Adaptivity method **9**
3.1 Mesh as a graph . . . 9
3.2 Diffusion-based mesh deformation . . . 9
3.3 PDE-dependent training objective . . . 9
3.4 Training and inference distinction . . . 10

4 Implementation and reproducibility **10**

5 Replication protocol **11**
5.1 From the validation ladder to the paper-exact protocol . . . 11
5.2 Models compared . . . 12
5.3 The seven experiments . . . 12
5.4 Reconstruction of the Burgers rollout protocol . . . 13
5.5 Seeds, aggregation and traceability . . . 13

6 Numerical results **13**
6.1 Evaluation protocol and measurement semantics . . . 14
6.2 Poisson on the unit square . . . 15
6.3 Poisson on a convex polygon . . . 16
6.4 Poisson on non-convex domains: L-shape and H-shape . . . 17
6.5 Burgers equation, fixed 10-step horizon . . . 18
6.6 Burgers equation, rollout evaluation . . . 19
6.7 Navier–Stokes flow past a cylinder . . . 20
6.8 Mesh relocation timing . . . 21
6.9 Summary of the replication status . . . 22

7 Discussion **22**
7.1 Analysis of the experiments . . . 22
7.1.1 Mechanisms behind the G-Adaptive advantage . . . 22
7.1.2 Non-convex geometries and Monge–Ampère failure . . . 23
7.1.3 Hyperbolic dynamics in the Burgers equation . . . 23
7.1.4 Difformer training instability (Navier–Stokes) . . . 23
7.2 What remains open . . . 24

8 Conclusions and future work **24**

List of Figures

1 Poisson square: G-Adapt mesh movement and mesh test samples. 15

2 Poisson square: FEM solutions on the same 25 test samples computed on the uniform, MA-adapted and G-Adapt-adapted meshes. 16

3 Poisson on a convex polygon: mesh movement and final G-Adapt meshes. 16

5 Poisson on non-convex domains 18

6 Burgers, 10-step horizon, G-Adapt: mesh movement on a representative test sample 19

7 Burgers rollout, G-Adapt: mesh movement on a representative test sample. 20

8 Navier–Stokes, G-Adapt: mesh movement on a representative test sample 21

List of Tables

1 Seven experiments of the final campain 13

2 Poisson on the unit square (paper Table 1). Our values: mean ± std across training seeds of the per-run test-set mean (100 test samples per run). 15

3 Poisson on a convex polygon (paper Table 1). Same conventions as Table 2. 16

4 Poisson on non-convex domains (paper Table 5) 17

5 Burgers on the unit square, 10-step horizon (paper Table 2). Same conventions as Table 2. 18

6 Burgers rollout 19

7 Navier–Stokes flow past a cylinde 20

8 Mean mesh relocation time per test sample 21

9 Replication status per experiment 22

Code Repository

GitHub Repository - Mesh Optimization via Scientific ML

1 Introduction

Finite element methods (FEMs) are among the standard numerical tools for approximating solutions of partial differential equations (PDEs). Their main advantages are robustness, a well-developed mathematical theory, and availability in mature software ecosystems such as Firedrake [4]. Nevertheless, the accuracy obtained for a fixed computational budget depends critically on the mesh. When a PDE solution contains localized structures, high curvature regions or sharp transitions, a uniform mesh may waste degrees of freedom in regions where the solution is smooth, and remain under-resolved where the approximation error is largest.

Adaptive mesh strategies address this issue by allocating resolution where it is most useful. In *h-adaptivity*, cells are refined or coarsened, changing the topology and the number of degrees of freedom. In *r-adaptivity*, the number of mesh nodes and their connectivity are kept fixed, but the node coordinates are moved. The latter approach is attractive for time-dependent simulations and for software pipelines where preserving data structures is valuable. However, it is also difficult: one must move the nodes enough to improve the finite element error, while avoiding tangled elements and poorly conditioned meshes.

Classical relocation methods often use a monitor function, typically related to curvature or an interpolation error estimate, and then solve a separate meshing equation. A prominent example is the Monge–Ampère approach, where the mesh movement is described through an optimal-transport-inspired potential [2, 3]. The reference paper considered in this project, *G-Adaptivity: optimised graph-based mesh relocation for finite element methods* [1], proposes a different strategy. Instead of solving the classical meshing equation, a graph neural network (GNN) predicts the moved mesh. The network is trained using a PDE-dependent objective, namely the finite element error of the numerical solution on the moved mesh, together with a regularization term designed to preserve mesh quality.

The aim of this project is not to design a new mesh relocation method from scratch. Rather, the goal is to understand, run and document a replication of the G-Adaptivity pipeline. The work started from the Poisson benchmark on a square domain and was progressively extended until it covered the full experimental suite of the reference paper that is reproducible from the released code: Poisson problems on convex and non-convex geometries, the time-dependent Burgers equation, and a Navier–Stokes flow past a cylinder. This required several layers of work: studying the mathematical method, installing and configuring a Firedrake [4], PyTorch [7] and PyTorch-Geometric [8] environment, adapting the code to the current cluster software stack, constructing a reliable execution workflow, defining progressively more faithful experiment configurations, debugging the loss computation and output-saving pipeline, and finally running a multi-seed campaign of 73 jobs with the paper-exact protocol.

The report is organized as follows. Section 2 introduces the finite element and moving mesh background. Section 3 summarizes the reference G-Adaptivity method. Section 4 describes the codebase and configuration system, the cluster workflow, the code patches, the debugging steps, and also provides a link to the GitHub repository MeshOpt-via-ScientificML, which contains the full code, patches, and working environment used to obtain the replicated results. Section 5 presents the replication protocol, from the initial validation ladder to the paper-exact configurations. Section 6 presents the final multi-seed results for all seven experiments and compares them with the reference paper. The final sections discuss limitations and future work.

2 Mathematical and numerical background

This section recalls the numerical ingredients needed for the replication: the Poisson model problem, its finite element discretization, the role of mesh relocation, and the classical Monge–Ampère baseline against which the learned mesh mover is compared.

2.1 The Poisson model problem

The first benchmark considered by Rowbottom et al. [1] is the Poisson equation on a two-dimensional domain. The simplest instance is the square case,

$$\Omega = [0,1]^2.$$

The model problem can be written as

$$-\Delta u = f \quad \text{in } \Omega, \qquad u = g \quad \text{on } \partial\Omega. \tag{1}$$

In the experiments, the forcing and boundary data are associated with Gaussian-type fields. Concretely, each PDE instance is generated from a combination of Gaussian components,

$$f(z) = \sum_{k=1}^{K} a_k \exp\left(-\frac{\|z - c_k\|^2}{2\sigma_k^2}\right), \tag{2}$$

where the number of components K, the amplitudes a_k, the centres c_k and the widths σ_k are sampled randomly. In the configuration files this randomization is controlled by the dataset family `data_type: randg_mix`, which activates random Gaussian data, and by the key `num_gauss_range`: a value such as `[2, 3]` means that each instance contains either two or three Gaussian components, so that $K \in \{2, 3\}$ for that run. This choice is useful because Gaussian fields can generate localized regions of high curvature while remaining smooth enough to provide stable reference solutions. Such localized features make the problem a natural test case for adaptive mesh relocation.

The objective is not only to solve (1), but to compare how the finite element error changes when the same PDE instance is solved on different meshes:

(i) the original regular or computational mesh;

(ii) a mesh produced by a classical Monge–Ampère relocation method;

(iii) a mesh produced by a trained learned mesh mover.

2.2 Weak formulation and finite element discretization

For simplicity, consider homogeneous Dirichlet data throughout this section. The corresponding weak form is: find $u \in H_0^1(\Omega)$ such that

$$\int_\Omega \nabla u \cdot \nabla v \, \mathrm{d}x = \int_\Omega f v \, \mathrm{d}x \qquad \forall v \in H_0^1(\Omega). \tag{3}$$

Let $\mathcal{T}_Z$ be a triangulation of Ω, with mesh nodes collected in

$$Z = \{z^{(i)}\}_{i=1}^{N_z}.$$

The finite element space on this mesh is denoted by $V_Z \subset H_0^1(\Omega)$. With piecewise linear elements, the discrete approximation $U_Z \in V_Z$ satisfies

$$\int_\Omega \nabla U_Z \cdot \nabla v_Z \, \mathrm{d}x = \int_\Omega f v_Z \, \mathrm{d}x \qquad \forall v_Z \in V_Z. \tag{4}$$

The dependence on Z is essential: changing the mesh nodes changes the finite element space, the assembled matrices and the final approximation error.

The reference paper [1] focuses on reducing a squared L^2 error of the form

$$E(Z, U_Z) = \|U_Z - u\|_{L^2(\Omega)}^2. \tag{5}$$

In practical computations, the exact solution u is replaced by a high-resolution reference solution or by a projection onto a sufficiently accurate finite element space. The reported error reduction compares the finite element error on an adapted mesh to the error on the original mesh.

2.3 Mesh relocation and r-adaptivity

In relocation-based adaptivity, a fixed computational mesh is mapped to a physical mesh. Let Ω_{C} denote the computational domain and Ω_{P} the physical domain. A deformation map

$$F : \Omega_{\mathrm{C}} \to \Omega_{\mathrm{P}} \tag{6}$$

assigns each computational coordinate ξ to a physical coordinate $z = F(\xi)$. If $\xi^{(i)}$ are the fixed computational nodes, then

$$z^{(i)} = F(\xi^{(i)}). \tag{7}$$

The connectivity of the mesh remains fixed, while the physical node positions change.

The goal of r-adaptivity is therefore to find node positions Z that reduce the finite element error while preserving an acceptable mesh. In ideal form, one would like to solve

$$\min_{Z} E(Z, U_Z), \tag{8}$$

subject to mesh validity constraints. Directly solving (8) is expensive because each candidate mesh requires a finite element computation, and the dependence of U_Z on Z is implicit through the discrete PDE. Classical methods therefore introduce proxy objectives based on interpolation error or monitor functions.

2.4 Classical Monge–Ampère mesh movement

A widely used class of moving mesh methods relies on equidistributing a monitor function [3, 2] $m(z)$ over the mesh. The monitor is chosen to be large where the solution is difficult to approximate, for example where a curvature indicator is large. In Monge–Ampère-based methods, the deformation map is related to a scalar mesh potential ϕ by

$$F(\xi) = \nabla_{\xi}\phi(\xi). \tag{9}$$

The Jacobian of the map is then the Hessian $H(\phi)$, and the equidistribution condition leads formally to the Monge–Ampère equation

$$m(z)\,|H(\phi)| = m(\nabla\phi)\,|H(\phi)| = \theta, \tag{10}$$

where θ is constant. This approach has strong mathematical motivation, and the convexity of the potential helps avoid mesh tangling. It is used as the main classical baseline in the G-Adaptivity paper [1].

A concrete monitor used both in the reference paper and in the inspected codebase is the gradient-based monitor

$$m(U_Z) = 1 + 5\,\frac{|\nabla U_Z|}{\max_{\Omega}|\nabla U_Z|}, \tag{11}$$

which is large where the solution varies rapidly and equal to one in flat regions, so that nodes are attracted towards high-gradient zones. The corresponding Hessian-based variant (`monitor_type: monitor_hessian`) replaces the gradient norm with curvature information; in either case the strength of the monitor is controlled by `monitor_alpha`, whose typical value for two-dimensional problems is 5.0, consistent with the factor in (11).

Moreover, solving the meshing PDE adds computational cost, and the method has structural failure modes. Standard Monge–Ampère solvers do not converge on the Navier–Stokes cylinder domain: the method constructs the mesh map as $F = \nabla P$ with P convex, which by Brenier's theorem is a homeomorphism between the (simply connected) computational domain and the physical domain, and since simple connectedness is a topological invariant, no such convex-potential map can exist onto a domain with an interior hole. Separately, on non-convex domains the relocated mesh can leave the domain or degenerate near reentrant corners. The central question motivating G-Adaptivity is whether one can learn a fast mesh relocation map that directly improves the finite element error and remains robust in these regimes.

2.5 Other reference problems

For completeness, we recall that the reference study benchmarks G-Adaptivity on a larger set of problems than the Poisson square case introduced above. On the Poisson problem itself, Rowbottom et al. [1] also consider a convex polygonal domain and five additional non-convex geometries, including L-shaped and H-shaped domains with reentrant corners, precisely the setting in which the Monge–Ampère baseline degrades, as noted above. Two further PDEs test entirely different physics: the viscous Burgers' equation for a vector velocity field u,

$$\partial_t u + (u \cdot \nabla)u - \nu \nabla^2 u = 0, \tag{12}$$

solved on a square domain, and the incompressible Navier–Stokes equations

$$\partial_t u + (u \cdot \nabla)u - \nu \nabla^2 u + \nabla p = f, \qquad \nabla \cdot u = 0, \tag{13}$$

solved on the flow past a cylinder domain.

2.6 Error reduction and mesh quality metrics

The main numerical metric used in the report is relative error reduction. If E_0 denotes the finite element error on the original mesh and E_{adapt} the error on an adapted mesh, then

$$\text{ER}(\%) = \frac{E_0 - E_{\text{adapt}}}{E_0}\, 100. \tag{14}$$

Larger values are better: a positive ER means that the adapted mesh has reduced the finite element error with respect to the original mesh. This is the same convention used in the reference paper [1].

It is important to note that this convention differs in sign from the one used in the raw output of the repository. The code writes an error-reduction column defined as

$$\frac{E_{\text{adapt}} - E_0}{E_0}\, 100 = -\,\text{ER}(\%), \tag{15}$$

so that in the raw CSV files a *negative* value indicates an improvement. To avoid any ambiguity, all error-reduction values reported in this document follow the convention of (14), in which a larger positive value is better; the raw repository outputs were sign-flipped accordingly before being tabulated. This makes the numbers in this report directly comparable with the reference paper.
A second metric is the time required to relocate the mesh. This is important because the promise of a trained GNN is fast online inference: training may be expensive, but once the model is trained, generating a new mesh should be much cheaper than solving a classical nonlinear meshing problem. A caveat that had to be resolved during the project concerns the units: the raw timing CSVs written by the pipeline record all times in *seconds* (this was verified directly in the instrumented code, which uses differences of `time.time()` calls), whereas the tables of the reference paper are in milliseconds. All timing values reported in this document have been converted to milliseconds so that they can be compared directly with the paper; the precise semantics of each timer (what is included in the measured interval, and when it is measured) are documented in Section 6.1, because they turn out to be essential for a fair comparison.

Finally, mesh quality is monitored through indicators such as minimum angle, scaled Jacobian and aspect ratio. In the reference paper, the aspect ratio is used as a summary measure of mesh quality; a smaller aspect ratio generally indicates a more regular mesh, while a single-digit aspect ratio is usually acceptable for the reported experiments.

3 The G-Adaptivity method

We now summarize the reference method of Rowbottom et al. [1] at the level needed to understand the implementation and the numerical experiments reproduced in this project.

3.1 Mesh as a graph

A triangular mesh naturally defines a graph. The nodes of the graph are the mesh vertices, and two graph nodes are connected if the corresponding mesh vertices belong to a common cell. If $G = (V, E)$ denotes this graph, node features can be stored in a matrix

$$X \in \mathbb{R}^{N_z \times d_0}.$$

Typical features include the current node coordinates, information about the coarse finite element solution, and curvature or Hessian-based quantities. The GNN acts on these features and outputs relocated coordinates.

This graph representation is well suited to adaptive meshes because the graph connectivity encodes the local neighborhood structure of the finite element mesh. A message-passing or diffusion-based network can therefore move a node using information from nearby elements and solution features.

3.2 Diffusion-based mesh deformation

The reference model uses a diffusion-like graph dynamics inspired by graph neural diffusion ideas [5, 1]. In continuous pseudo-time, we can write:

$$\frac{\partial X(t)}{\partial t} = \big(A_\theta(X(t)) - I\big)X(t), \tag{16}$$

where A_θ is a learned attention matrix and I is the identity. The attention matrix is row-stochastic, so each node update resembles a weighted average over graph neighbors. In the discrete implementation, the update rule is formulated using an explicit Euler-type scheme,

$$X^{k+1} = X^k + \Delta t \,\big(A_\theta(X^k) - I\big)X^k. \tag{17}$$

The paper setting, which is also the one executed in the final campaign of this project, uses four GNN layers, called *diffformer blocks* in the paper. Each block performs 32 explicit Euler steps of size 0.1, with each step sequentially moving the mesh nodes once. The relocated coordinates are then passed to the next layer or, eventually, to the network's output.

The importance of this structure is twofold. First, it physically constrains how the network learns: rather than acting as a black-box model that outputs arbitrary node coordinates, the GNN behaves like a trainable moving mesh equation. Second, the diffusion dynamics help preserve mesh regularity. Intuitively, if a node is moved toward a convex combination of neighboring positions, it is less likely to cross mesh lines or create inverted elements than under an unconstrained displacement field.

3.3 PDE-dependent training objective

The central novelty of G-Adaptivity, as introduced by Rowbottom et al. [1], is the training objective. Earlier machine-learning approaches to mesh movement often learned to simply imitate meshes generated by classical methods [6, 9]. In contrast, the G-adaptive model is trained to reduce the finite element error itself. Formally, if M_θ denotes the GNN mesh mover, the adapted mesh is

$$Z_\theta = M_\theta(Z_0, \text{features}), \tag{18}$$

where Z_0 is the initial mesh. The FEM evaluation on Z_θ produces U_{Z_θ}, and the objective contains

$$E(Z_\theta, U_{Z_\theta}). \tag{19}$$

The gradient of this quantity with respect to mesh coordinates is obtained through Firedrake adjoint tools. Backpropagation then propagates this gradient through the GNN parameters.

In practice, the training loss also contains a mesh regularization term. A representative form is

$$\mathcal{L}_\theta = E(Z_\theta, U_{Z_\theta}) + \lambda\, \mathcal{L}_{\text{equi}}(Z_\theta), \tag{20}$$

where the term $\mathcal{L}_{\text{equi}}$ penalizes poor equidistribution using λ as a regularization weight.

The $\mathcal{L}_{\text{equi}}$ term is essential because minimizing only the PDE error can produce meshes that overconcentrate nodes or degrade element quality. This equidistribution loss is based on cell areas or volumes and on a Hessian-based monitor. For a triangular cell K, let $|K|$ denote its area and let m_K be a cell-wise evaluation of the monitor discussed in Section 2.4, and N_T the number of elements. The regularizer compares the products $|K|m_K$ across cells and penalizes deviations from their average. In simplified notation,

$$\mathcal{L}_{\text{equi}} \approx \frac{1}{N_T} \sum_{K \in \mathcal{T}} \left(|K| m_K - \frac{1}{N_T} \sum_{K' \in \mathcal{T}} |K'| m_{K'} \right)^2. \tag{21}$$

3.4 Training and inference distinction

The Firedrake adjoint computation is needed during training, when gradients of the FEM error are required. Once training is complete, inference only requires a forward pass through the GNN to generate the relocated mesh. This distinction explains the main computational promise of the method: expensive PDE-aware training is performed offline, while online mesh relocation can be much faster than solving the classical Monge–Ampère meshing problem for every new PDE instance. The timing results of Section 6.8 quantify this trade-off on our hardware.

4 Implementation and reproducibility

This section summarizes how the G-Adaptivity pipeline was configured, executed and debugged for the replication. The implementation-level detail (repository layout, full configuration reference, cluster workflow, PBS templates and the complete debugging log) is documented in the project repository MeshOpt-via-ScientificML. Here we retain only what is needed to understand the protocol and the results of Sections 5 and 6.

Codebase and configuration: The work builds on the public `g-adaptivity` codebase released with Rowbottom et al. [1]; all references to "the repository" and "the G-adaptive implementation" denote that method and code, integrated into our NAPDE project repository. The entry point is the `run_pipeline.py` script, which reads a YAML configuration, builds or loads data, trains or loads the model, evaluates it and logs results. Default settings in `base_config.yaml` are overwritten by experiment-specific `repro_*` files, which encode the paper-exact protocol for each benchmark and its learned-model variants (G-Adapt, UM2N-G, UM2N). A practical caveat, important for correct execution, is that the mapping between paper parameters and YAML keys is not one-to-one: the diffusion deformer actually reads `grand_diffusion`, `grand_diffusion_steps` and `grand_step_size`, whereas the similarly named `GNN_diffusion` key used in the base configuration is legacy and ignored by the model code. The `repro_*` files therefore set these keys explicitly. A representative configuration file and the full key reference are given in the README of MeshOpt-via-ScientificML repo.

Cluster workflow: All 73 production runs were executed through PBS batch scripts on the CPU queue of the PoliMI DMAT student cluster; the login node was used only for submission, monitoring and file transfer. Persistent files, environments and results live under `/work`, while I/O-intensive jobs stage data on node-local `/scratch_local` and copy outputs back upon completion. Runs are made batch-friendly and deterministic by pinning BLAS/OpenMP thread counts to one, enforcing non-interactive Matplotlib plotting and isolating per-run cache directories. Because the nodes are shared and each optimisation step performs a Firedrake adjoint solve, absolute walltimes are load-dependent and training-dominated; the relocation times reported in Section 6 refer only to evaluation-time inference.
The full software stack, hardware description and PBS templates are listed in the README.

Patches and debugging: The changes made in the codebase fall into three distinct categories, which are reported separately in the README: implemented compatibility patches, cluster workflow adaptations, and diagnostic modifications.
Two implemented patches were necessary for the campaign: a PyVista cell-quality compatibility block (to compute mesh-quality metrics under the installed API), and a persistent output-saving patch that writes all raw CSV files, summaries, plots and the exact `config_used.yaml` under `RESULTS_DIR`, so that every number in this report is traceable to a job ID.
An optional gradient-clipping flag (`-grad_clip_norm`) was added to study a seed-dependent instability of the diffusion deformer on the Navier–Stokes and Burgers benchmarks; runs using it are excluded from the headline averages to stay aligned with the paper protocol.
The most instructive episode was a hidden PDE-loss failure: early runs produced a smooth, tiny training loss ($\sim 10^{-6}$) while the validation FEM error failed to improve.

A bare `except` clause in `run_GNN.py` was silently discarding a Firedrake/PyTorch incompatibility (a `'Cofunction' object has no attribute 'vector'` error) on essentially every datapoint, collapsing the objective from $L_{\text{PDE}} + \lambda L_{\text{reg}}$ to just λL_{reg}: the network was optimising mesh geometry alone and learning nothing about the PDE. Exposing the exception, fixing the underlying Firedrake object and verifying the gradient connectivity of both loss terms restored the intended objective. Together with a separate observation that the equidistribution loss used a mean rather than the sum specified in the paper (which depressed the resulting loss magnitude), this shows that a smooth loss curve is necessary but not sufficient evidence of correctness. The complete before/after code, the diagnostic checklist and the patch-status table are documented in the README. We regard this debugging work as a central methodological contribution of the project.

5 Replication protocol

5.1 From the validation ladder to the paper-exact protocol

The reference paper reports a compact set of hyperparameters, but the repository exposes many YAML keys and implementation-specific options. Additionally, cluster software versions, Firedrake compatibility and output handling had to be validated before running expensive configurations. The project therefore first went through a progressive ladder of cheap configurations: a smoke test (2 training samples, one epoch, 8×8 mesh) to validate the environment and the pipeline end-to-end, followed by intermediate runs with reduced dataset sizes (75–275 samples), reduced mesh ranges (11×11–15×15) and 20–50 epochs.
These runs served their purpose, as they exposed the PyVista incompatibility, the output-saving needs and, most importantly, the hidden PDE-loss failure of Section 4. However, their numerical outputs are deliberately not reported, as they used a protocol (mesh range, epochs, dataset size) that differs from that of the paper. Moreover, before the loss bug was fixed, some of them did not optimize the intended objective at all.

All results in Section 6 were produced in the final campaign: 73 runs executed with the `repro_*` configurations, which reproduce the protocol of the reference paper one-to-one , with multiple independent training seeds per learned model. A run is considered part of the campaign only if it completed successfully, matched the reference dataset sizes and met or exceeded the paper's epoch count.

5.2 Models compared

Following the reference paper, four mesh-relocation methods are compared on each benchmark:

MA: The classical Monge–Ampère baseline, implemented through the `MongeAmpereMover` of the `movement` package (v0.3) with the Hessian-based monitor (11) (`monitor_alpha` = 5). The solver parameters follow the codebase defaults: relaxation method with `rtol` $= 10^{-4}$ and at most 500 iterations for the square, polygon and Burgers domains; quasi-Newton with `rtol` $= 5 \times 10^{-3}$ and at most 200 iterations for the L-shape and H-shape. Since MA is deterministic given the dataset it involves no training and no seed.

G-Adapt: The G-Adaptivity model of the paper: transformer-based feature encoder plus the diffusion-based "Diffformer" deformer (4 blocks, 32 explicit Euler steps per block, step size 0.1). It is trained with the regularized PDE loss (20) with $\lambda = 1$ for 300 epochs using the Adam algorithm, learning rate 10^{-3} and batch size 1.

UM2N-G: A modified version of G-Adapt utilizing the same pipeline and regularized PDE loss, but with the deformer replaced by the GAT-based architecture of UM2N [9]. The same training protocol as G-Adapt is used.

UM2N: The original UM2N method [9], trained from scratch with its own loss (`UM2N_loss`) for 1000 epochs, as described in the paper. Because it is designed to learn to imitate classical remeshing methods, it fundamentally requires ground-truth target meshes (generated via MA) for training. For non-convex domains, where MA fails and cannot produce valid training data, the authors of the paper bypassed training and instead evaluated a pretrained UM2N model sourced directly from the original repository provided by Zhang et al. [9]. Because the integration of these weights is not supported by the released `g-adaptivity` codebase, those results could not be replicated in our campaign (see Section 6.9).

The paper also reports a direct mesh-optimisation upper baseline (DirectOpt), which performs gradient descent on the mesh coordinates of each test instance individually. Because its cost is two to three orders of magnitude above all other methods ($\sim$126 s per sample on the paper's hardware) it was not included in our campaign; the paper values are quoted for context where relevant.

5.3 The seven experiments

Table 1 summarizes the seven experiments of the final campaign, which mirror the experimental suite of the reference paper.

Experiment	Geometry / data	Train/test	Meshes (train; test)	Seeds (G-Adap / UM2N-G)
Poisson square	unit square, `randg_mix`	100/100	15^2, 20^2; 12^2–23^2	5 / 5
Poisson polygon	convex polygon, `randg`	100/100	fixed `.msh`	5 / 5
Poisson L-shape	L-shape, `randg`	100/100	fixed `.msh`	5 / 5
Poisson H-shape	H-shape, `randg`	100/100	fixed `.msh`	5 / 5
Burgers, 10 steps	unit square, `randg_mix`	100/100	15^2, 20^2; 12^2–23^2	5 / 5
Burgers, rollout	unit square, `randg_mix`	100/100	15^2, 20^2; 12^2–23^2	5 / 5
Navier–Stokes cylinder	cylinder domain, `RBF`	25/50	fixed `.msh`	6 / 5

Table 1: The seven experiments of the final campaign (G = G-Adapt, U-G = UM2N-G, U = UM2N). All learned models use 300 training epochs (1000 for UM2N), Adam with learning rate 10^{-3}, batch size 1, and the Hessian monitor; G-Adapt and UM2N-G use the regularized PDE loss with $\lambda = 1$. For the square-domain experiments training uses 15×15 and 20×20 meshes and evaluation spans 12×12 to 23×23, with the error measured against a high-resolution reference (evaluation grid 100×100, degree-4 reference space), exactly as in the paper.

5.4 Reconstruction of the Burgers rollout protocol

The rollout evaluation of the paper does not exist in the released code and had to be reconstructed from the text: ten trajectories are advanced for 20 time steps, the mesh is re-adapted every 2 steps, and the reported metric is the mean error reduction per 2-step block. The delicate point is how consecutive blocks are chained. Our first implementation transported each method's own coarse state (its predicted solution on the sparse, adapted mesh) through the remeshing steps, a process that requires interpolating the solution from one coarse mesh to another. With this approach, the interpolation error at each remeshing step accumulates and leads to different results from those shown in the reference paper. So, in the proposed protocol, each block restarts from the state computed by using the high-resolution reference solution and measures how much the adapted mesh reduces the error of the next two steps; this reproduces the qualitative structure of the paper's table. The residual ambiguity, whether the remeshing input is the reference state or the method's own transported state, is not resolvable from the outside and turns out to matter for one model (Section 6.6).

5.5 Seeds, aggregation and traceability

Every learned-model entry in Section 6 is the mean over independent training seeds (5 seeds, 6 for Navier–Stokes G-Adapt, due to bimodal error reduction outputs) of the per-run mean over the test set; the quoted uncertainty is the standard deviation *across seeds*. MA entries carry no seed variability.

6 Numerical results

This section reports the final results of the 73-run campaign, one subsection per experiment, followed by the timing comparison. Each table reports our multi-seed mean ± standard deviation across seeds next to the corresponding value of the reference paper [1]. All error reductions follow the positive-is-better convention of (14). Moreover, two systematic remarks apply to all subsequent tables:

- **Dataset Regeneration**: Our absolute MA values differ from the reference paper because the random datasets were regenerated using the released codebase. Because MA is deterministic and depends solely on the individual problem instances, its performance on our regenerated datasets naturally differs from the results provided in the paper. It is worth noting that, had the exact problem instances used by the authors been available to us, our MA values would coincide exactly with those reported in the paper: MA is a deterministic solver, so its output depends only on the specific mesh, forcing term and boundary data it is given, not on who generates them. The `g-adaptivity` repository notes that generating datasets directly is computationally expensive due to the large number of FEM solves involved, and for this reason provides a set of precomputed training and test datasets via Zenodo. However, this precomputed release does not cover the full range of configurations required to replicate the paper's protocol; for most of the experiments reported here the datasets,,

- **UM2N Baseline**: When trained from scratch with its original loss (1000 epochs), the UM2N model failed to converge to a successful mesh mover in our setup, consistently yielding negative error reductions. This aligns with the reference paper's own caveat that "further tuning of the training may be required." Additionally, as detailed in Section 5.2, we were unable to evaluate UM2N on domains where MA fails. For these reasons, the UM2N rows have been omitted from most tables.

Finally, a detailed theoretical analysis of all of the following results is deferred to Section 7.

6.1 Evaluation protocol and measurement semantics

For each test instance, the evaluation compares the original regular mesh, the MA-adapted mesh and the mesh produced by each learned model; the PDE is solved on each mesh and the L^2 error is computed against a high-resolution reference solution. Error reduction, relocation time and aspect ratio are recorded per sample.

Comparing relocation times requires careful interpretation, as the execution times for the learned models and the Monge–Ampère (MA) baseline are measured differently:

- **Learned models**: the recorded time captures only the neural network's forward pass (feature encoder and deformer), measured inside the `forward()` function in the code. It explicitly excludes model loading, dataset construction, and the final FEM computation on the relocated mesh. Because this is measured live during each run, the times exhibit seed-to-seed variance reflecting load fluctuations on cluster nodes.

- **MA**: the recorded time covers the `mover.move()` call of the Monge–Ampère solver. This intrinsically includes the cost of intermediate FEM computations required to update the monitor function. Additionally, for non-rollout benchmarks, MA meshes are generated and cached only once, during the dataset creation process. Consequently, the MA time is a single static measurement with zero variance across all runs of the same experiment, regardless of the seed.

- **Burgers rollout**: this is the only benchmark where both MA and the learned models are timed live in the same job at each remeshing step, thus providing the cleanest timing comparison.

- **Navier–Stokes**: the MA solver requires a simply connected domain to converge, and thus does not converge on a domain with a cylinder inside (which is multiply connected). Its timing column is therefore omitted (reported as "NA").

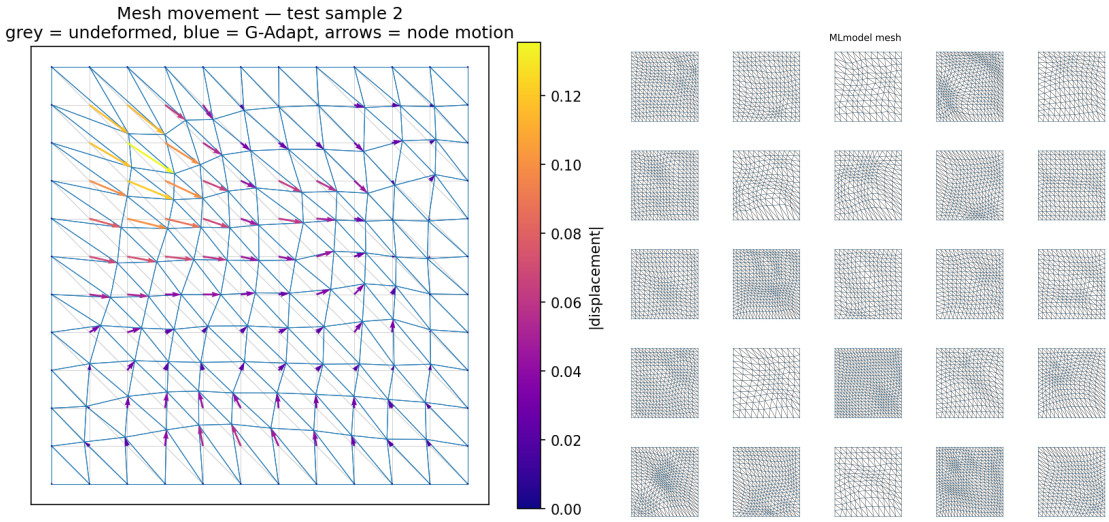
Note that all raw times are recorded in seconds by the pipeline and have been converted to milliseconds in the tables.

6.2 Poisson on the unit square

	Error reduction (%)		Aspect ratio		
Model	Ours	Paper	Ours	Paper	n seeds
MA	15.87	12.69 ± 0.00	2.05	2.11 ± 0.00	–
G-Adapt	22.44 ± 1.66	21.01 ± 0.33	2.71 ± 0.17	2.92 ± 0.03	5
UM2N-G	18.41 ± 3.44	16.40 ± 2.65	2.40 ± 0.33	2.61 ± 0.17	5
UM2N	-3.40	6.83 ± 1.10	3.01	1.99 ± 0.03	1
DirectOpt	—	27.40 ± 0.00	—	33.99 ± 0.00	–

Table 2: Poisson on the unit square (paper Table 1). Our values: mean ± std across training seeds of the per-run test-set mean (100 test samples per run).

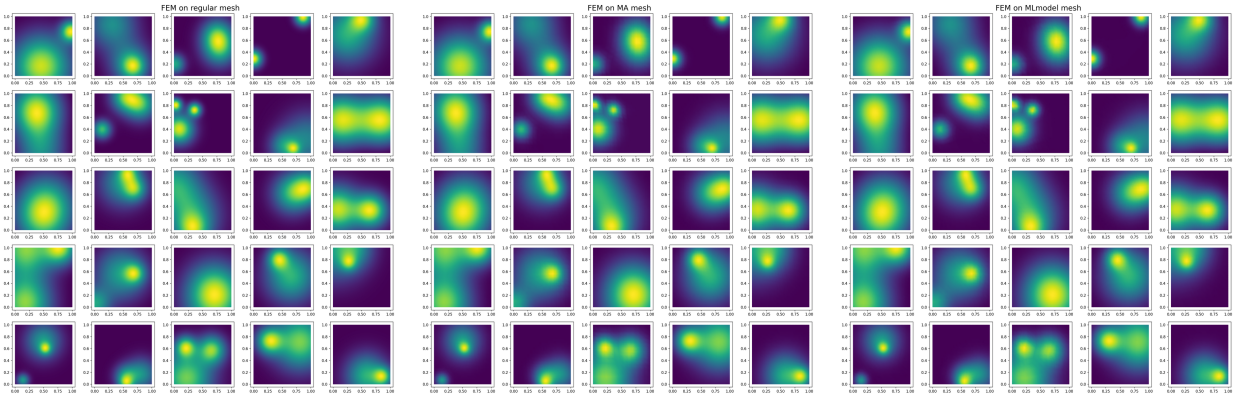
The central claim of the paper is reproduced quantitatively. G-Adapt achieves $22.44 \pm 1.66\%$ error reduction against the paper's $21.01 \pm 0.33\%$, which is compatible within one standard deviation, and clearly outperforms the MA baseline, with a G-Adapt$-$MA gap of $+6.6$ points against $+8.3$ in the paper. The ordering UM2N $<$ MA $<$ UM2N-G $<$ G-Adapt $<$ DirectOpt is identical to the paper's, and the aspect ratios agree closely.



(a) G-adapt mesh movement on a representative test sample

(b) G-adapt meshes on 25 test samples

Figure 1: Poisson square: G-Adapt mesh movement and mesh test samples. In (a) Grey lines indicate initial uniform mesh; blue, G-Adapt mesh; arrows show node displacements, with color mapping the displacement magnitude. Nodes concentrate around the Gaussian features of the forcing term.



(a) Uniform mesh. (b) MA mesh. (c) G-Adapt mesh.

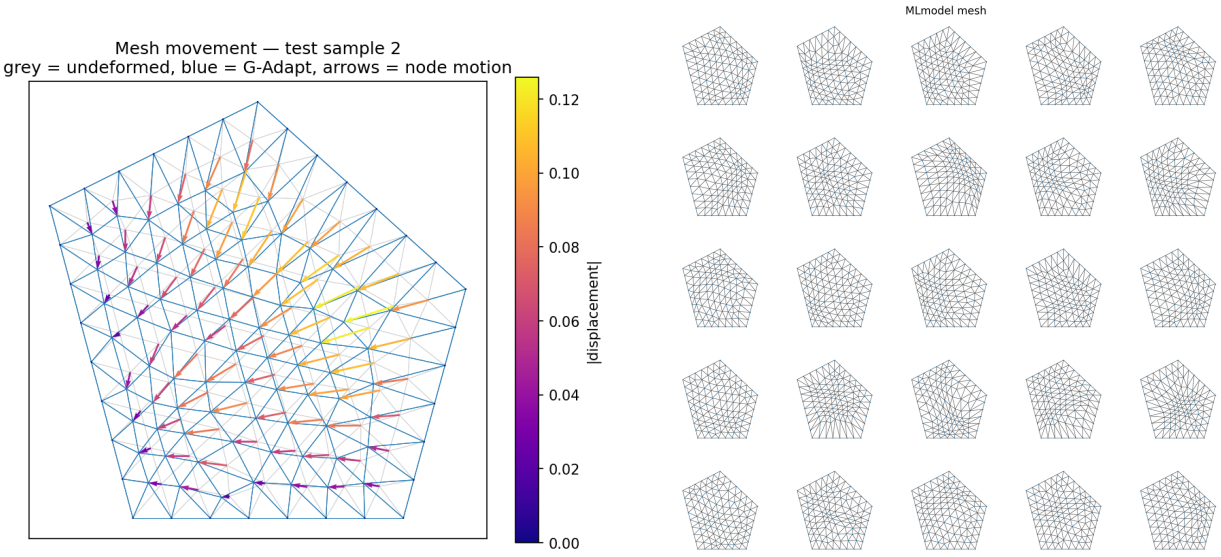
Figure 2: Poisson square: FEM solutions on the same 25 test samples computed on the uniform, MA-adapted and G-Adapt-adapted meshes.

6.3 Poisson on a convex polygon

	Error reduction (%)		Aspect ratio		
Model	Ours	Paper	Ours	Paper	n seeds
MA	12.71	10.97 ± 0.00	1.92	1.95 ± 0.00	–
G-Adapt	15.70 ± 0.80	16.84 ± 0.10	1.88 ± 0.04	1.86 ± 0.02	5
UM2N-G	15.03 ± 0.74	15.00 ± 0.13	1.82 ± 0.17	1.88 ± 0.03	5
UM2N	-3.55	3.12 ± 0.38	2.00	1.66 ± 0.03	1
DirectOpt	—	20.51 ± 0.00	—	2.07 ± 0.00	–

Table 3: Poisson on a convex polygon (paper Table 1). Same conventions as Table 2.

The polygon benchmark represents the closest quantitative replication in our campaign: UM2N-G matches the reference paper to the second decimal (15.03 vs. 15.00), G-Adapt is within 1.1 points, and aspect ratios are nearly identical. These results perfectly reproduce the paper's model ranking and verify that G-Adapt outperforms the MA baseline (+3.0%, compared to +5.9% in the paper).



(a) Mesh movement, representative sample. (b) G-Adapt meshes, first 25 test samples.

Figure 3: Poisson on a convex polygon: mesh movement and final G-Adapt meshes.

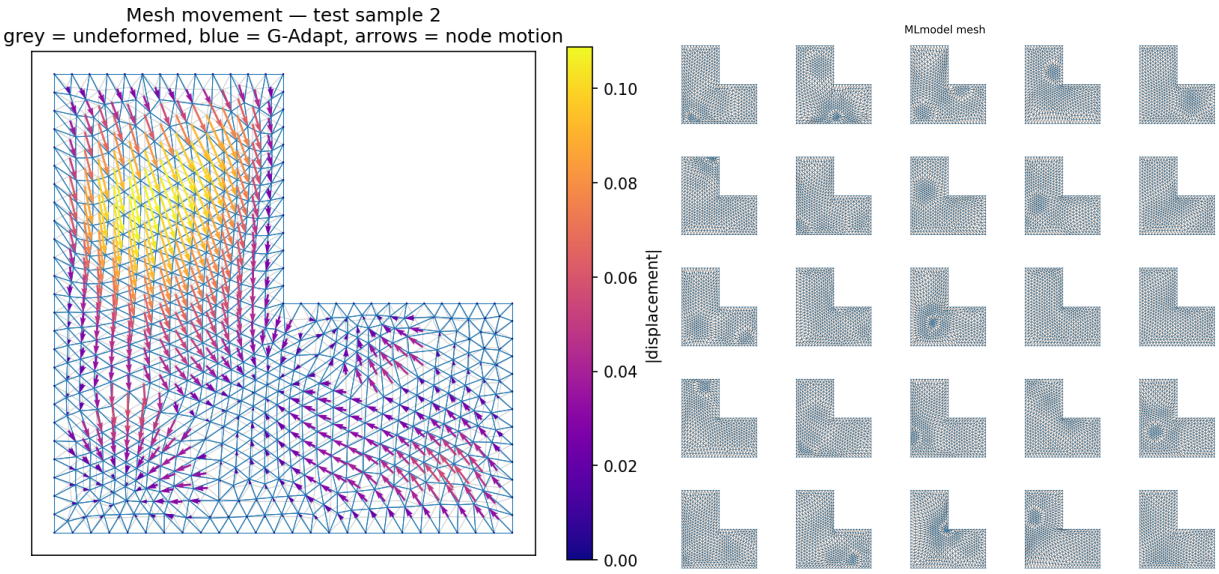
6.4 Poisson on non-convex domains: L-shape and H-shape

		Error reduction (%)		Aspect (ours)	n seeds
Domain	Model	Ours	Paper		
L-shape	MA	-64.18	-89.40 ± 0.00	77.9	–
	G-Adapt	16.48 ± 0.28	16.25 ± 0.25	$21.2^{(*)}$	5
	UM2N-G	15.78 ± 0.14	13.94 ± 1.18	2.68 ± 0.25	5
H-shape	MA	-108.31	-108.31 ± 0.00	248.0	–
	G-Adapt	3.98 ± 0.36	7.51 ± 0.00	1.38 ± 0.03	5
	UM2N-G	-0.44 ± 0.07	-0.92 ± 0.00	1.37 ± 0.00	5

Table 4: Poisson on non-convex domains (paper Table 5). The paper does not report aspect ratios for these geometries. $^{(*)}$The L-shape G-Adapt aspect mean is dominated by a single seed with a near-degenerate element at the reentrant corner (aspect 90); the other four seeds lie between 3.2 and 5.2.

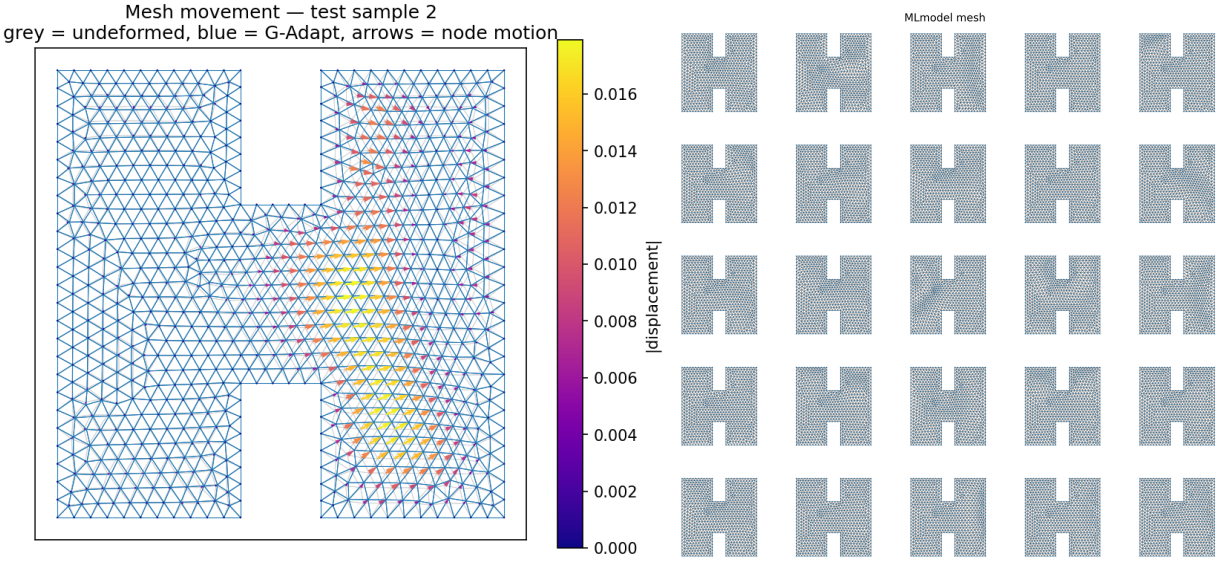
Results on both non-convex geometries perfectly match the paper's qualitative claims: the MA baseline fails, worsening the error and producing extreme aspect ratios, while G-Adapt consistently improves the mesh. On the L-shape, G-Adapt achieves a $16.48 \pm 0.28\%$ error reduction, aligning particularly closely with the paper's $16.25 \pm 0.25\%$. For the H-shape, our MA baseline matches the paper exactly (-108.31%). This perfect alignment occurs because the H-shape is the only configuration where the authors themselves provided, through their GitHub repository, a dataset that exactly matched the parameters declared in the paper.
However, G-Adapt significantly underperforms compared to the paper on this specific geometry ($3.98 \pm 0.36\%$ vs. $7.51 \pm 0.00\%$). Because UM2N-G performs as expected (-0.44% vs. -0.92%), this performance gap is isolated exclusively to G-Adapt. A plausible explanation for the discrepancy on the H-shape is that the paper's figure for this geometry does not come from an average over several seeds. The reference value is reported as 16.25 ± 0.25 elsewhere but as 7.51 ± 0.00 here, and the same null spread appears for UM2N-G on the same geometry, namely -0.92 ± 0.00. A learned model trained from a random initialisation cannot exhibit exactly zero variance across independent seeds, and indeed everywhere else in the paper the learned models carry an appreciable spread: on the L-shape, for instance, 16.25 ± 0.25 for G-Adapt and 13.94 ± 1.18 for UM2N-G.
The only entries for which a null standard deviation is legitimate are the deterministic baselines such as MA. We therefore suspect that the H-shape column reports a single training run rather than a mean over five seeds, and plausibly the most favourable one. Our own value, 3.98 ± 0.36, is by construction the mean over five independently seeded runs, which is precisely why it carries a nonzero standard deviation; selecting the best of five runs would necessarily yield a value above such a mean.

This does not fully account for the gap, but it suggests that part of it stems from a difference in reporting protocol rather than from a difference in method performance, and it makes the comparison on this geometry less directly meaningful than on the L-shape, where both figures are averages over several seeds and agree closely.



(a) L-shape: mesh movement.

(b) L-shape: G-Adapt meshes, 25 samples.



(a) H-shape: mesh movement.

(b) H-shape: G-Adapt meshes, 25 samples.

Figure 5: Poisson on non-convex domains: G-Adapt mesh movement and resulting meshes for the L-shape and H-shape. The learned model adapts the mesh without the corner degeneracies that break the MA baseline.

6.5 Burgers equation, fixed 10-step horizon

	Error reduction (%)		Aspect ratio		
Model	Ours	Paper	Ours	Paper	n seeds
MA	9.26	12.28 ± 0.00	2.01	1.99 ± 0.00	–
G-Adapt	19.62 ± 9.30	21.66 ± 3.13	4.09 ± 0.64	2.82 ± 0.06	5
UM2N-G	14.19 ± 4.48	16.38 ± 2.85	2.17 ± 0.22	1.83 ± 0.10	5

Table 5: Burgers on the unit square, 10-step horizon (paper Table 2). Same conventions as Table 2.

Both learned models roughly match the paper's performance and reproduce the expected ranking (MA < UM2N-G < G-Adapt). However, our results show around three times more variance across seeds than the paper reports (with G-Adapt ranging from 9.5% to 32.8%). As discussed in Section 7, this stems from an optimisation instability. We verified this by running two control tests with gradient clipping, which safely avoided low-performing seeds and yielded stable error reductions of 21.2% and 33.7%.

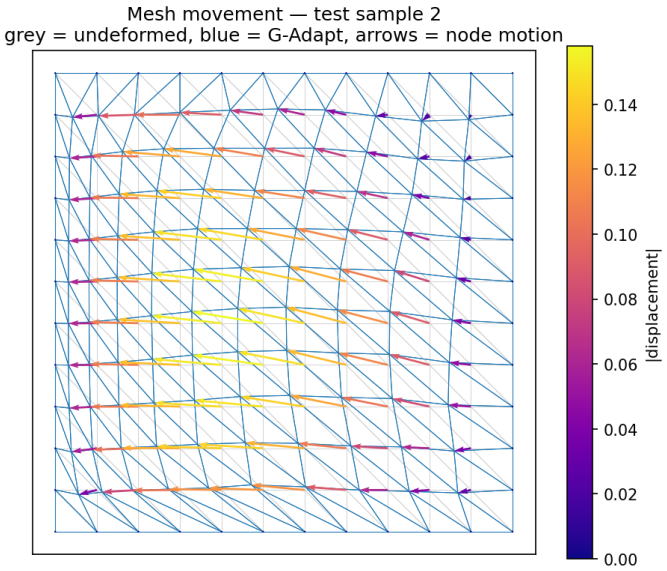


Figure 6: Burgers, 10-step horizon, G-Adapt: mesh movement on a representative test sample. Because the reference codebase lacks visualization support for the physical solutions of vector-valued PDEs such as Burgers and Navier-Stokes, we utilize a custom script to plot the node displacements directly.

6.6 Burgers equation, rollout evaluation

	Error reduction (%)		Aspect ratio		
Model	Ours	Paper	Ours	Paper	n seeds
MA	22.36	25.78 ± 0.00	2.04	1.99 ± 0.00	–
G-Adapt	22.44 ± 1.00	27.17 ± 0.34	2.80 ± 0.13	2.85 ± 0.12	5
UM2N-G	23.13 ± 1.25	-2.33 ± 3.73	2.74 ± 0.14	2.83 ± 0.11	5

Table 6: Burgers rollout: 10 trajectories × 20 time steps, remeshing every 2 steps, mean error reduction per block (paper Table 2), under the reconstructed protocol of Section 5.4.

Because the rollout evaluation code was not released, the protocol had to be reconstructed directly from the paper's text (see Section 5.4). Under this protocol, G-Adapt essentially ties with MA (22.44% vs. 22.36%), confirming the paper's conclusion that the learned model matches classical methods in rollout scenarios. However, UM2N-G achieved a positive $+23.13 \pm 1.25\%$ error reduction in our runs, which contradicts the severe performance collapse ($-2.33 \pm 3.73\%$) reported in the paper.

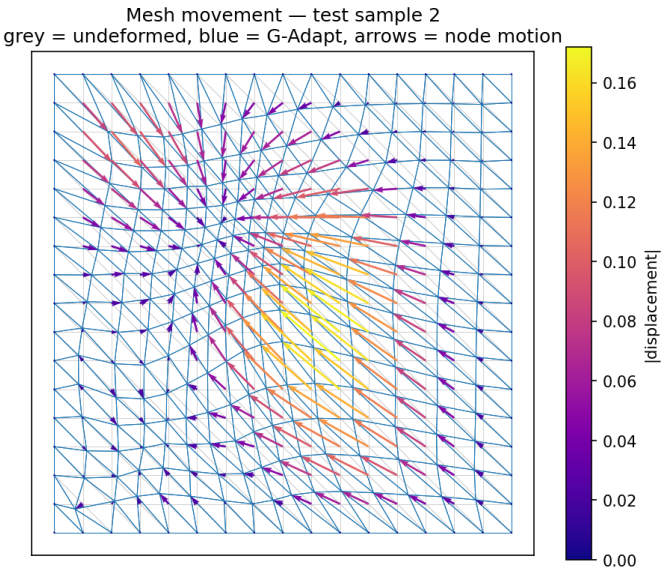


Figure 7: Burgers rollout, G-Adapt: mesh movement on a representative test sample.

6.7 Navier–Stokes flow past a cylinder

	Error reduction (%)		Aspect ratio		
Model	Ours	Paper	Ours	Paper	n seeds
MA	n/a	NA	—	—	–
G-Adapt	21.84 ± 5.10	26.36 ± 1.37	2.73 ± 0.71	3.51 ± 0.81	6
UM2N-G	25.08 ± 1.51	25.55 ± 0.81	3.21 ± 0.35	2.32 ± 0.06	5

Table 7: Navier–Stokes flow past a cylinder (paper Table 3). MA is structurally inapplicable on the multiply connected domain (the paper reports "NA"; the code fixes the MA mesh to the uniform grid).

For the Navier-Stokes problem, the UM2N-G model achieved a stable mean of $25.08 \pm 1.51\%$, closely matching the paper. In contrast, G-Adapt's $21.84 \pm 5.10\%$ average masks a severe training instability: four seeds successfully reached the paper's benchmark ($\sim$26%), but two collapsed completely ($\sim$15–16%). As detailed in Section 7, applying gradient clipping (max norm 1.0) successfully prevented this collapse, yielding stable error reductions of $\{19.17\%, 25.17\%, 25.37\%\}$ on three separate control seeds.

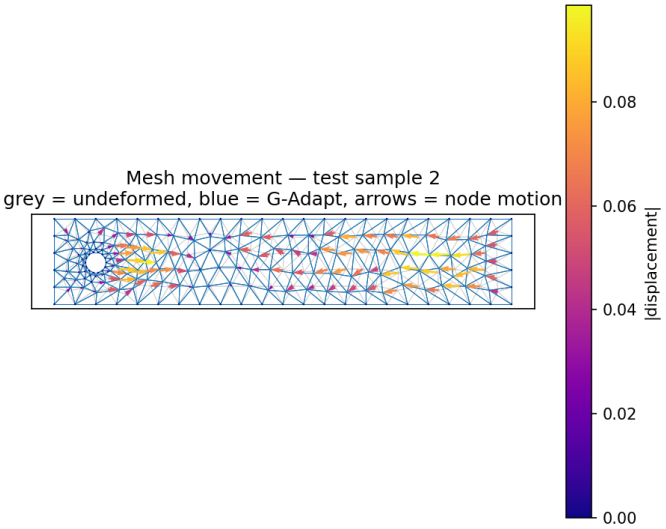


Figure 8: Navier–Stokes, G-Adapt: mesh movement on a representative test sample. Nodes concentrate in the wake of the cylinder.

6.8 Mesh relocation timing

Table 8 compares mesh relocation times across all experiments, in milliseconds, following the measurement semantics detailed in Section 6.1.

	MA (ms)		G-Adapt (ms)		UM2N-G (ms)	
Experiment	Ours	Paper	Ours	Paper	Ours	Paper
Poisson square	56 681*	3 780	185 ± 43	88	49 ± 1	30
Poisson polygon	50 971*	4 446	110 ± 30	55	21 ± 3	16
Poisson L-shape	4 452*	1 306	309 ± 31	160	144 ± 58	88
Poisson H-shape	550*	550	388 ± 38	158	167 ± 33	59
Burgers, 10 steps	39 891*	11 566	164 ± 8	93	60 ± 10	41
Burgers, rollout	19 197 (†)	18 884	435 ± 250	717	244 ± 44	316
Navier–Stokes	n/a	NA	142 ± 47	49	37 ± 6	30

Table 8: Mean mesh relocation time per test sample (per remeshing block for the Burgers rollout), in milliseconds. (Note: our UM2N times, omitted for brevity, are 70 ms on the square and 46 ms on the polygon, compared to the paper's 70 ms and 36 ms). *Single cached measurement (see Section 6.1). (†)The rollout MA mean is inflated by a single job executed on a heavily loaded node (88.6 s/block); the remaining nine runs produced results between 9.6 and 14.9 s/block, with median ≈ 11.6 s.

The results confirm the paper's claims: UM2N-G is the fastest, followed closely by G-Adapt, while MA is drastically slower. In the Burgers rollout, that is the only benchmark that, as explained in Section 6.1, measures MA and the learned models simultaneously under the exact same conditions, an MA/G-Adapt ratio of 27× is recorded, effectively matching the 26× speedup reported in the paper.

6.9 Summary of the replication status

Experiment	Status	Notes
Poisson square	Reproduced	ER compatible within 1 std; ordering and aspect ratios match.
Poisson polygon	Reproduced	Closest match of the campaign (UM2N-G to the 2nd decimal).
Poisson L-shape	Reproduced	MA failure regime confirmed; G-Adapt matches paper.
Poisson H-shape	Partial	MA identical to paper (-108.31); G-Adapt gap 3.98 vs 7.51 unexplained (Section 6.4).
Burgers, 10 steps	Reproduced	Means compatible; across-seed variance $3\times$ the paper's (optimisation instability).
Burgers, rollout	Reproduced$^{(\dagger)}$	Protocol reconstructed; G-Adapt ties MA as in the paper. UM2N-G sign discrepancy open ($+23$ vs -2).
Navier–Stokes	Reproduced$^{(\dagger)}$	UM2N-G matches paper; G-Adapt bimodal across seeds (Diffformer instability, mitigated by clipping). MA "NA" confirmed.
Timing	Reproduced	Ordering and rollout ratio match; absolute MA times slower on our hardware.
UM2N rows (pretrained)	Not replicable	Requires pretrained UM2N weights not shipped with the code.
DirectOpt rows	Not attempted	Cost two–three orders of magnitude above all other methods.

Table 9: Replication status per experiment. $^{(\dagger)}$Reproduced up to a documented protocol ambiguity or with a documented deviation in seed statistics.

7 Discussion

This section interprets the experimental findings of Section 6: it explains the mechanisms behind the reproduced claims and the anomalies, and then collects the results whose interpretation remains open.

7.1 Analysis of the experiments

7.1.1 Mechanisms behind the G-Adaptive advantage

The central result of the replication, that the learned model matches or exceeds the MA baseline in error reduction on the convex Poisson benchmarks, follows from the different objectives the two methods actually minimise. Classical MA relocation does not target the finite element error (5) at all: it equidistributes a monitor function, a construction motivated by interpolation-error theory, which controls the FEM error only through an upper bound with problem-dependent constants. Two distinct sources of slack are therefore introduced. First, minimising an upper bound is not equivalent to minimising the quantity it bounds, so the MA mesh need not be optimal even when the meshing equation is solved exactly. Second, the monitor itself is a heuristic surrogate for where the error concentrates, built from gradient or curvature information rather than from the error of the discrete solution. G-Adaptivity removes both layers: the Firedrake adjoint supplies gradients of the assembled FEM error with respect to the node coordinates, so the network optimises the target quantity directly.

The efficiency result has an equally simple explanation. The expensive PDE-aware computation is confined to training: at inference the model performs a forward pass through the diffusion deformer, with no nonlinear meshing problem to solve, whereas MA must solve one for every new instance. The speed-up recorded in Table 8 is therefore a structural property of the approach rather than an implementation detail, which is why the ordering of the methods reproduces the

paper's even though our absolute MA times are slower (Section 6.8).Its magnitude, however, is strongly geometry-dependent: it exceeds two orders of magnitude on the square and the polygon, where the MA relaxation requires many iterations, and falls to near parity on the H-shape, where the classical solver converges quickly and the learned model must displace a comparatively large mesh. Its Validity, finally, is not obtained for free but is enforced by the equidistribution regulariser together with the diffusion-based architecture, whose non-tangling property replaces the convexity argument that underpins the classical method: the same substitution that explains the robustness advantage discussed next.

7.1.2 Non-convex geometries and Monge–Ampère failure

The failure of the MA baseline on the non-convex geometries (L-shape and H-shape) stems directly from the regularity theory of fully nonlinear PDEs. In the classical MA formulation the mesh deformation is defined through the gradient of a scalar potential, and a strictly convex domain is required for the equation to retain ellipticity. Reentrant corners violate this requirement, breaking ellipticity and producing degenerate mappings; numerically, this manifests as the severe mesh tangling and extreme aspect ratios observed in our experiments. Because its learned dynamics are not constrained by the global convexity of a potential, G-Adaptivity bypasses this limitation, which is the source of its robustness advantage on these domains.
The H-shape also explains why G-Adaptivity itself reaches only a modest error reduction there. The central horizontal "bridge" acts as a narrow geometric bottleneck that strictly bounds the admissible node displacement: any aggressive attempt to concentrate nodes within it pushes elements against the boundary and degrades the aspect ratio. The space of valid, non-degenerate deformations is therefore intrinsically smaller than on the other domains, which caps the achievable improvement for any relocation-based method. The residual gap between our value and the paper's is a separate matter and is treated in Section 7.2.

7.1.3 Hyperbolic dynamics in the Burgers equation

The larger mesh deformations and the wider across-seed variance seen in the Burgers benchmark follow from the advection-dominated, nonlinear hyperbolic nature of the equation. Unlike the elliptic Poisson problem, Burgers develops and transports steep wave fronts, so reducing the error requires the mesh to track sharp, moving gradients, which demands larger displacements. Since nonlinear advection maps different initial conditions into very different shock dynamics, the difficulty of the optimisation varies strongly from one problem instance to the next; this heterogeneity is the attenuated form of the same optimisation instability analysed next for Navier–Stokes, here softened by the larger training set.

7.1.4 Diffformer training instability (Navier–Stokes)

The Navier–Stokes experiment exposed a behaviour the reference paper does not report: a seed-dependent training instability of the diffusion-based deformer. The G-Adapt result is not normal variance around a mean but a bimodal outcome, with successful seeds matching the paper's level and the remaining ones stalling after an early upward jump of the training loss (about 35%) onto a permanent plateau. This is the signature of a single anomalous Adam step that ejects the weights from a good basin, from which fixed-rate optimisation never recovers.
Two controls confirm the diagnosis. Gradient clipping (max norm 1.0) removes the catastrophic collapse, and UM2N-G is stable across all seeds. Together these isolate the instability to the coupling between the Diffformer architecture and fixed-rate Adam on small datasets, rather than to the data or the pipeline. For a method whose value proposition is robust automation, this sensitivity to individual gradient steps is a practically relevant finding.

7.2 What remains open

The following results are not fully resolved by the replication:

(i) **The H-shape G-Adapt gap.** G-Adapt yields $3.98 \pm 0.36\%$ against the paper's $7.51 \pm 0.00\%$. The low variance of our runs makes seed noise an unlikely cause, and the exact agreement of the MA column confirms the datasets are identical, so the gap is not a dataset-regeneration effect. The paper's zero variance on this table suggests single-seed reporting, but this cannot be verified externally.

(ii) **The UM2N-G sign discrepancy (Burgers rollout).** We obtain $+23\%$ where the paper reports a degradation (-2%). As discussed in Section 5.4, this most plausibly stems from an ambiguity in the unreleased rollout evaluation regarding which state drives the remeshing at each step.

(iii) **The UM2N pretrained benchmarks.** These rows cannot be replicated because the pretrained weights are not shipped with the released G-Adaptivity repository.

(iv) **Absolute MA timing.** Our relative timing comparisons match the paper, but the absolute MA times on the convex domains are about an order of magnitude slower, consistent with single-threaded CPU-only execution on shared nodes.

(v) **Statistical power of the protocol.** The paper's protocol evaluates each model over only five seeds, which is too few to characterise the seed distribution reliably and leaves the averages sensitive to outliers, as we can see in the Navier–Stokes case. Rigorous robustness claims would require scaling the number of seeds.

8 Conclusions and future work

This report documented a replication of the G-Adaptivity method for finite element mesh relocation of Rowbottom et al. [1], covering the full set of experiments reproducible from the released code: four Poisson geometries, two Burgers protocols and a Navier–Stokes benchmark, through a 73-run, multi-seed campaign on the PoliMi cluster.

Experimental validation: The paper's central claims are confirmed. The learned G-adaptive model matches or outperforms the MA baseline in error reduction on all applicable benchmarks (*performance*); it succeeds on the non-convex and multiply connected domains where MA fails structurally (*robustness*); and inference relocates meshes one to two orders of magnitude faster than solving the MA equations (*efficiency*). Quantitatively, five of the seven experiments agree with the paper within seed dispersion, with the remaining offsets largely explained by the regeneration of the random datasets.

Methodological contributions: Beyond reproducing the numbers, the project produced findings and tools that the reference paper does not provide. We identified and fixed a silent PDE-loss failure in which the network stopped optimising the intended objective while the logged loss kept decreasing smoothly, a failure mode invisible from the training curve alone and transferable to any pipeline coupling an external differentiable solver to a neural network. We characterised a previously unreported, seed-dependent instability of the diffusion-based deformer and verified a working mitigation (gradient clipping). And we built a fully traceable batch-execution pipeline on HPC that preserves intermediate outputs, so that every number in this report is reproducible from an archived per-run record. These diagnostic and reproducibility aspects, detailed in Sections 4 and 7, we regard as a central outcome of the work rather than an incidental one.

Future work: The open points of Section 7.2 define the natural next steps: recovering the pretrained UM2N weights to complete the corresponding rows, and tuning the MA solver for a fairer absolute-timing comparison. More broadly, adopting gradient clipping (or a learning-rate schedule) as a standard protocol upgrade and scaling the number of seeds would put the robustness claims on firmer statistical ground, while extending the pipeline to three-dimensional benchmarks would round out the replication.

A proposed change of objective: A more substantive extension concerns the error functional itself. The squared L^2 error (5) used throughout this work weights all pointwise discrepancies equally. This choice differs from the classical baseline theory, where the interpolation-error estimates are naturally stated in the H^1 seminorm, and the monitors employed in practice, including the gradient and Hessian-based choices of Section 2.4, are built precisely from derivative information. Replacing the objective with the squared H^1 error, or with a weighted combination of the two, would make the loss explicitly sensitive to gradient accuracy and would concentrate nodes in regions of rapid variation by construction rather than as a side effect. The benchmarks where this should matter most are exactly those in which the present objective is weakest: the steep viscous fronts of Burgers' equation, and the boundary layer and stagnation-point singularity of the cylinder flow in the Navier-Stokes equations, where the solution error is small in L^2 but its gradient is not. The change is also inexpensive to implement, since only the assembled functional would be modified. Nonetheless, a gradient-sensitive loss rewards more aggressive deformation, so the balance against the equidistribution regulariser would have to be retuned, and the resulting meshes should be monitored for aspect-ratio degradation. Establishing whether the H^1 objective improves error reduction, and at what cost in mesh quality, is a natural next experiment.

References

[1] J. Rowbottom, G. Maierhofer, T. Deveney, E. Mueller, A. Paganini, K. Schratz, P. Liò, C.-B. Schönlieb and C. Budd. *G-Adaptivity: optimised graph-based mesh relocation for finite element methods*. Proceedings of the 42nd International Conference on Machine Learning, PMLR 267, 2025. Also available as arXiv:2407.04516.

[2] C. Budd, M. Cullen and E. Walsh. Monge–Ampère based moving mesh methods for numerical weather prediction, with applications to the Eady problem. *Journal of Computational Physics*, 236:247–270, 2013.

[3] W. Huang and R. D. Russell. *Adaptive Moving Mesh Methods*. Springer, 2011.

[4] D. A. Ham et al. Firedrake User Manual. Imperial College London, University of Oxford, Baylor University and University of Washington, 2023.

[5] B. Chamberlain, J. Rowbottom, M. Gorinova, M. Bronstein, S. Webb and E. Rossi. GRAND: Graph Neural Diffusion. Proceedings of the 38th International Conference on Machine Learning, 2021.

[6] W. Song, M. Zhang, J. G. Wallwork, J. Gao, Z. Tian, F. Sun, M. D. Piggott, J. Chen, Z. Shi, X. Chen and J. Wang. M2N: Mesh Movement Networks for PDE solvers. Advances in Neural Information Processing Systems, 2022.

[7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai and S. Chintala. PyTorch: an imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 2019.

[8] M. Fey and J. E. Lenssen. Fast graph representation learning with PyTorch Geometric. ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

[9] M. Zhang, C. Wang, S. Kramer, J. G. Wallwork, S. Li, J. Liu, X. Chen and M. D. Piggott. Towards universal mesh movement networks. Advances in Neural Information Processing Systems, 2024.